1. Desarrolla una **API REST en Node.js con Express y MongoDB** para gestionar el recurso **trabajador**.
   Cada trabajador debe tener los siguientes campos:

- nombre
- apellido
- cédula de identidad
- cargo
- departamento
- fecha de ingreso

Implementar los siguientes **endpoints**:

- GET /trabajador → Obtener todos los trabajadores
- POST /trabajador → Crear un nuevo trabajador
- PUT /trabajador/:id → Actualizar la información de un trabajador
- DELETE /trabajador/:id → Eliminar un trabajador

La base de datos será **MongoDB**.

Finalmente, despliega la aplicación en contenedores Docker utilizando **Docker Compose**, con un contenedor para la aplicación Node.js y otro contenedor para MongoDB.

Desarrollar una interfaz de consulta y manipulación de datos utilizando **GraphQL**, **Node.js** y **MongoDB** para gestionar el recurso "Trabajador", permitiendo consultas flexibles y una estructura de datos tipada.

1. Modelo de Datos

Cada trabajador debe estar definido en la base de datos (MongoDB) y en el esquema de GraphQL con los siguientes campos:

- **Nombre** (String)
- **Apellido** (String)
- **Cédula de identidad** (String, único)
- **Cargo** (String)
- **Departamento** (String)
- **Fecha de ingreso** (Date/String)

2. Requerimientos de GraphQL

Debes definir un esquema (Schema Definition Language - SDL) que soporte las siguientes operaciones:

Queries (Consultas)

- obtenerTrabajadores: Debe retornar una lista de todos los trabajadores registrados.
- obtenerTrabajador(id: ID!): Debe retornar los detalles de un trabajador específico por su ID.

Mutations (Mutaciones)

- crearTrabajador: Para registrar un nuevo trabajador (recibiendo los campos mencionados arriba como argumentos o mediante un input).
- actualizarTrabajador: Para modificar la información de un trabajador existente mediante su id.
- eliminarTrabajador: Para remover a un trabajador del sistema mediante su id.

3. Requerimientos Técnicos

1. **Servidor:** Utilizar Node.js con el framework Express y una implementación de servidor de GraphQL (como apollo-server-express o graphql-http).
2. **Base de Datos:** Utilizar MongoDB y la librería **Mongoose** para el modelado de datos y la conexión.
3. **Tipado:** El esquema de GraphQL debe reflejar correctamente los tipos de datos (ID, String, etc.) y marcar los campos obligatorios.

4. Despliegue con Docker

Finalmente, el sistema debe ser orquestado mediante contenedores utilizando **Docker Compose**, cumpliendo con lo siguiente:

- **Contenedor de Aplicación:** Basado en una imagen de Node.js que contenga el servidor GraphQL.
- **Contenedor de Base de Datos:** Utilizando la imagen oficial de MongoDB.

2. **Red:** Ambos contenedores deben estar en la misma red interna para comunicarse mediante el nombre del servicio.